pfi

Python for Fixed Income

COURSE 6 · NEURAL NETWORKS

Hidden Layers and Activations

WEEK 1 · DAY 3

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

What a hidden layer adds

Inputs, a hidden layer, an output

19 inputs

the card columns, scaled on the fit rows

hidden units

each a neuron: a total, then an activation

1 output

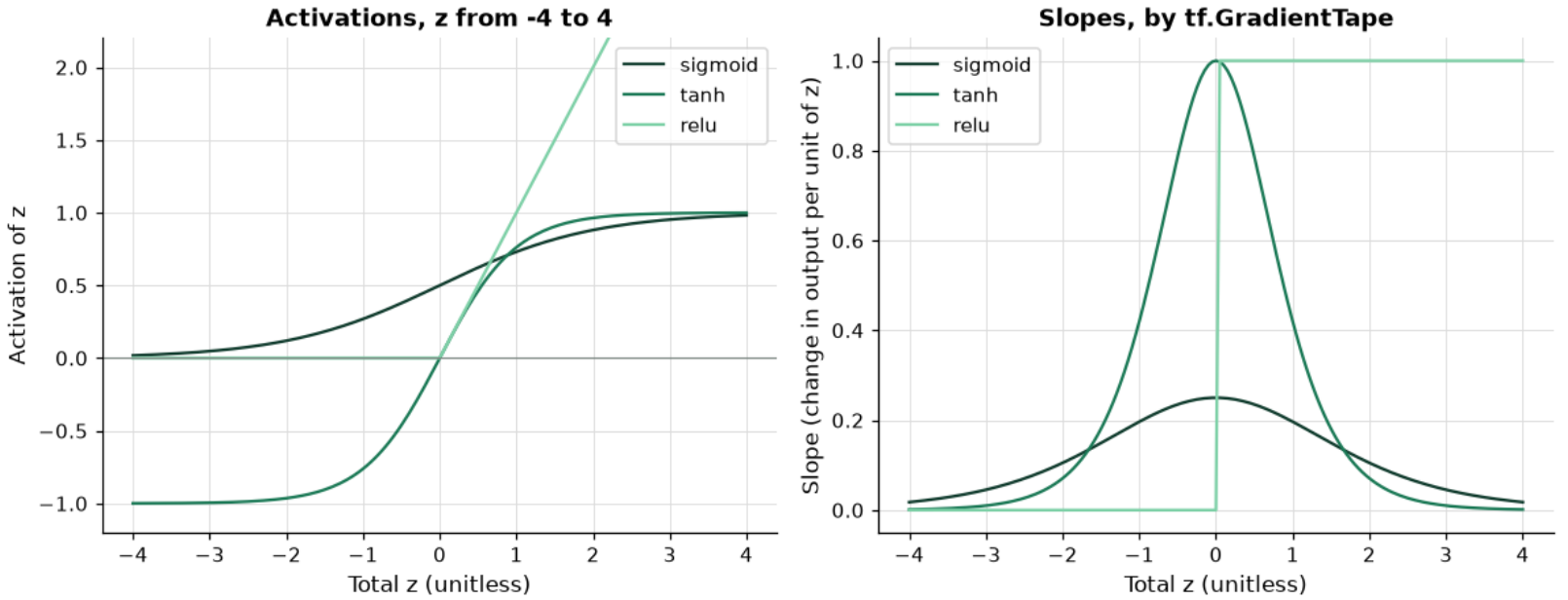
day 2's sigmoid neuron, on the hidden outputs

Today

1. Three activations and their slopes
2. Two hidden units by hand, and in Excel
3. forward_pass and parameter_count
4. A ladder of six networks
5. One report on the validation rows

One neuron draws a straight line before the sigmoid. Activations let a network bend.

Activations as functions



0.2500

sigmoid's largest slope

1.0000

tanh's largest slope

1.0000

ReLU's slope above 0

A hidden layer of two units, by hand

Weights (chosen by hand)	on PAY_0	on LIMIT_BAL	intercept
hidden unit 1	1.2	-0.3	-0.2
hidden unit 2	0.4	-0.9	0.1
output (on units 1, 2)	1.5	0.8	-1.6

First fit row

PAY_0 1.8078, limit -1.0596. ReLU unit 1 2.2873, unit 2 1.7768; output 0.9628 (tanh network 0.6513).

```
hidden = np.maximum(0.0, rows @ k + b)          # or np.tanh(rows @ k + b)
output = 1 / (1 + np.exp(-(hidden @ v + c)))     # equal to model.predict within 1e-6
```

Second row: unit 1 is 0.0000, a silent ReLU unit. 8,680 fit rows have at least one; a tanh unit is never silent.

forward_pass: the output layer by layer

```
for layer in model.layers:
    if Dropout: continue        # scoring time
    if not Dense: raise ValueError
    activation = layer.get_config()["activation"]
    kernel, bias = layer.get_weights()
    values = ACTIVATIONS[activation](values @ kernel + bias)
```

Three new tests

matches model.predict within 1e-5

passes Dropout through

refuses batch normalization

10

tests at the start

13

after forward_pass

15

after parameter_count

"linear" is what Keras stores for a Dense layer given no activation.

Counting parameters

```
a inputs into b units: a * b + b
```

Network	Sizes	Parameters
two units	2, 2, 1	9
Model 0	19, 1	20
Model 4	19, 32, 1	673
Model 5	19, 32, 16, 1	1,185

model.summary() of Model 5

Layer	Output shape	Param #
hidden_1	(None, 32)	640
hidden_2	(None, 16)	528
output	(None, 1)	17

```
Total params: 1,185
```

Shown once. Elsewhere: netkit.parameter_count and count_params(), one number each.

The ladder: one setting at a time

Model	Hidden layers	Parameters	Fit loss	Stopping loss
Model 0	one neuron	20	0.4617	0.4702
Model 1	16 sigmoid units	337	0.4380	0.4496
Model 2	16 tanh units	337	0.4253	0.4427
Model 3	16 ReLU units	337	0.4288	0.4449
Model 4	32 ReLU units	673	0.4256	0.4442
Model 5	32 and 16 ReLU units	1,185	0.4199	0.4448

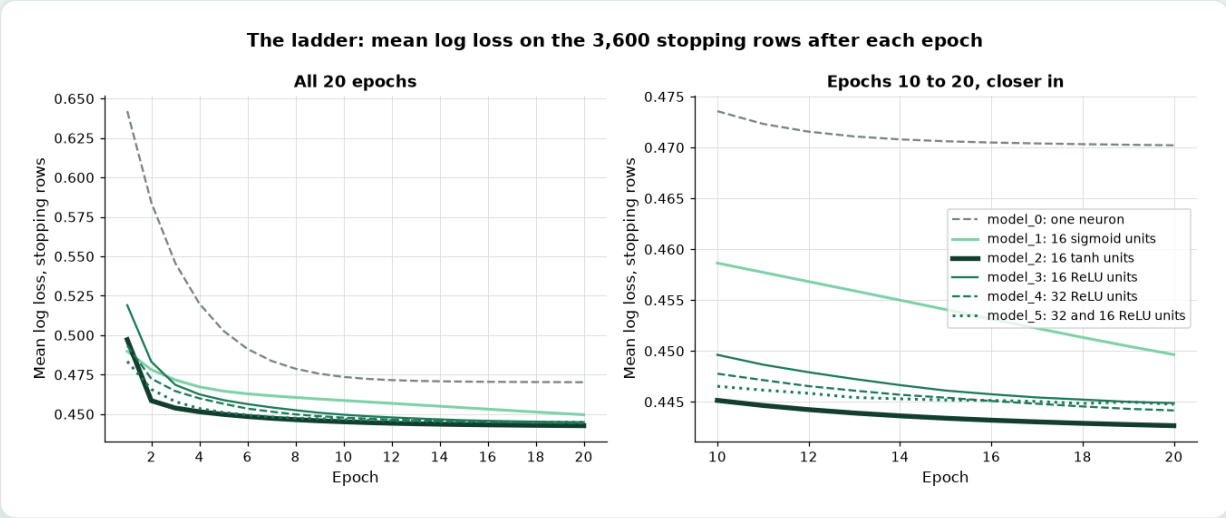
Chosen

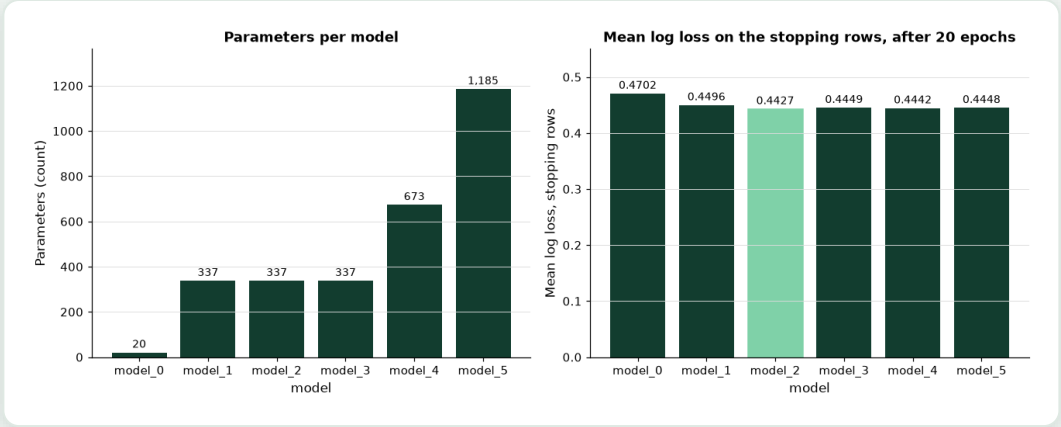
Model 2

lowest stopping loss among Models 1 to 5

20 epochs, batch 64, fitted on 14,400 rows; chosen on the 3,600 stopping rows. Models 2 to 5 lie within a seed's reach of each other.

The ladder, drawn





Every line lowest at epoch 20. Parameters from 20 to 1,185; stopping loss 0.4702 for Model 0, 0.4427 to 0.4449 for Models 2 to 5.

No activation, no hidden layer

```
keras.layers.Dense(32)                  # no activation= : a linear layer
w_one = k1 @ k2 @ k3                    # (19, 32) @ (32, 16) @ (16, 1) = (19, 1)
```

0.4702

linear stack, 1,185 parameters

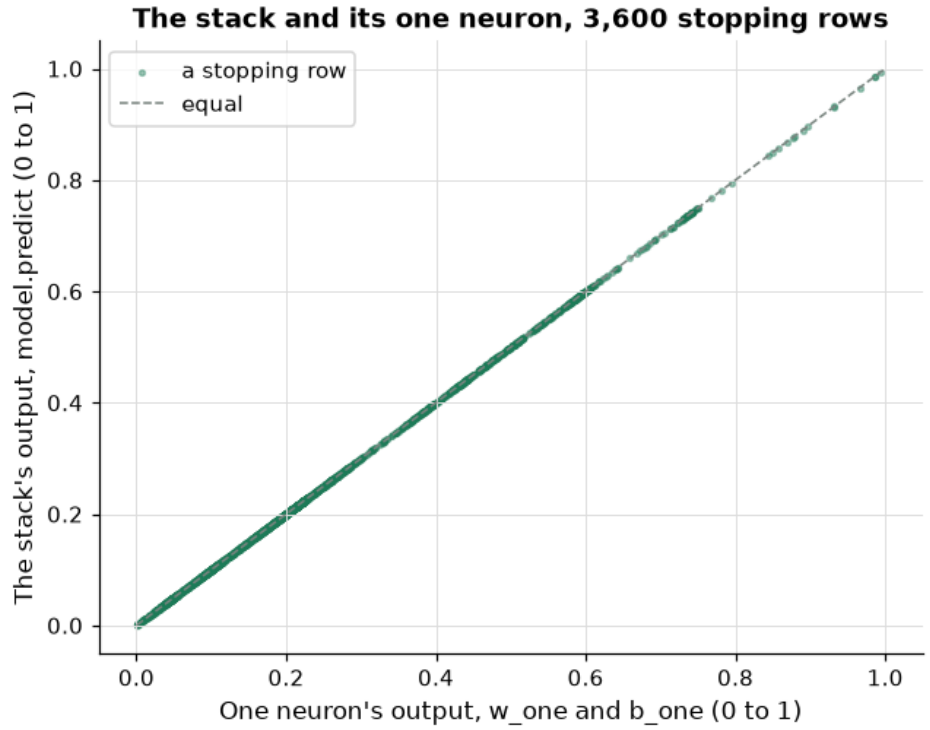
0.4702

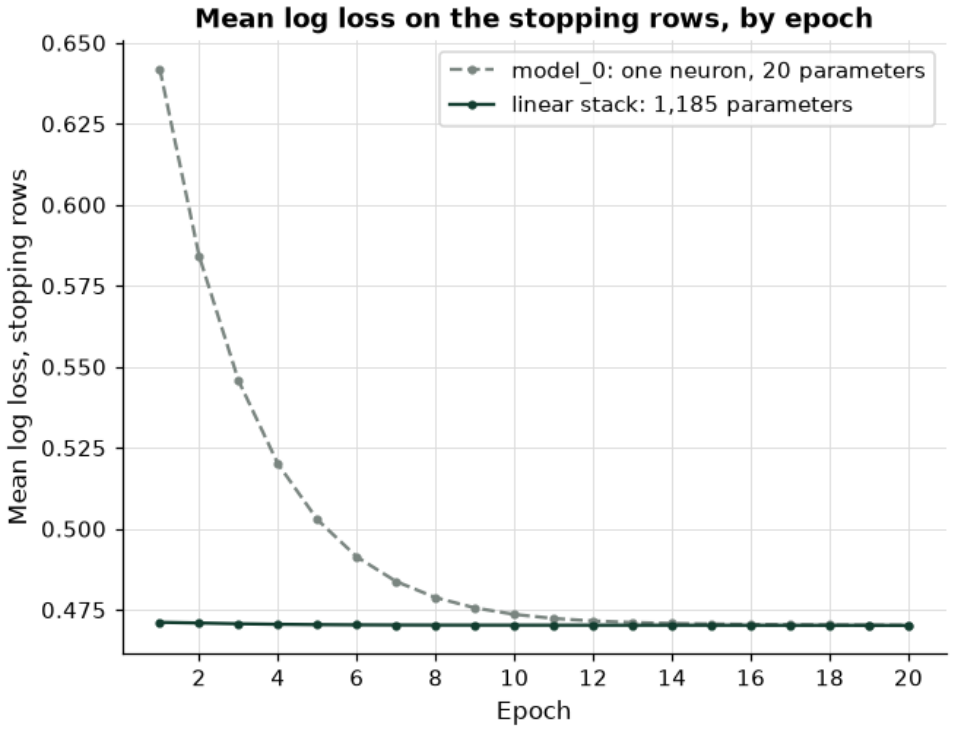
Model 0, one neuron, 20 parameters

One neuron with w_one and b_one equals the stack on every stopping row, within 1e-5.

The activation is what makes a hidden layer worth having. Keras does not warn you.

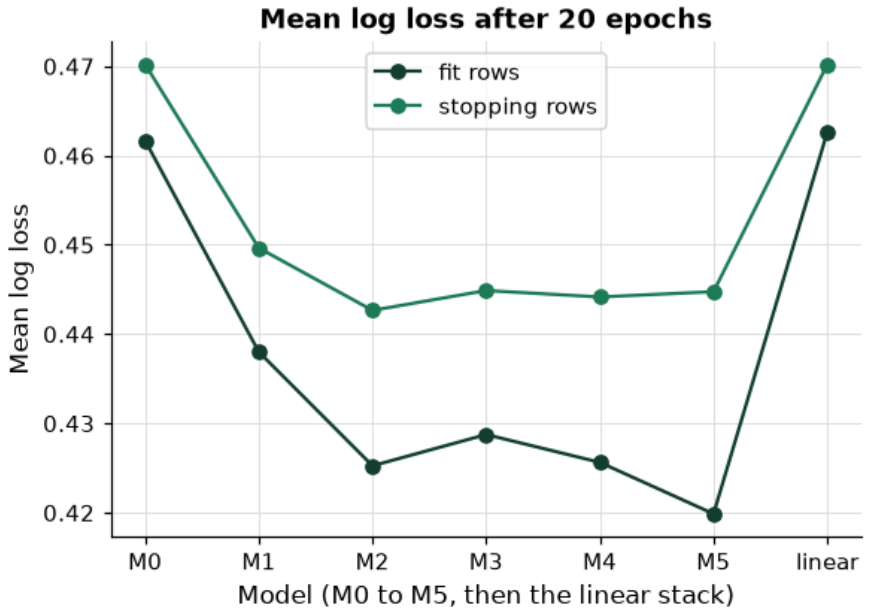
The stack, drawn

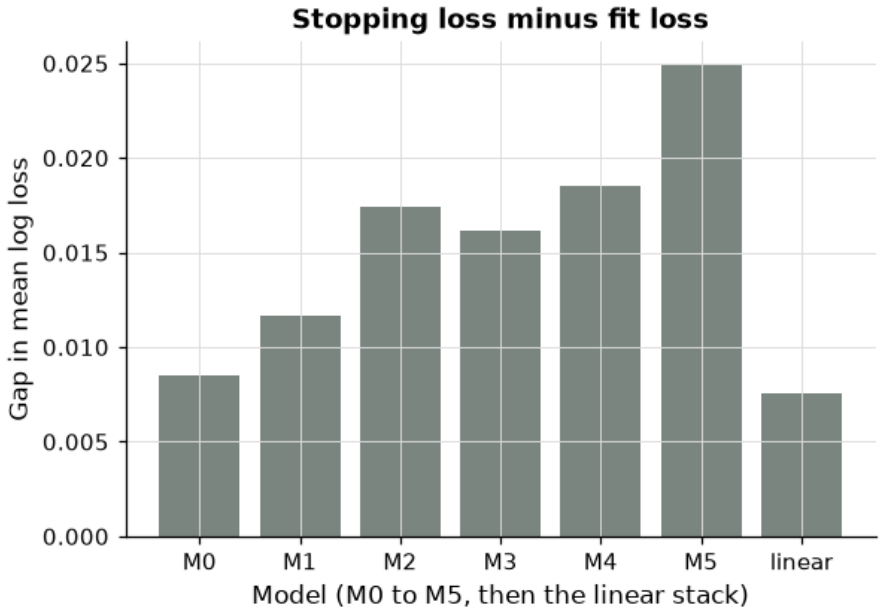




Every dot on the line. Both end at 0.4702: the stack got there faster, not further.

Width and depth on this file





Gap, stopping minus fit: 0.0161, 0.0185, 0.0249 for Models 3, 4, 5. Width helped a little here; depth did not.

The chosen network on the validation rows

0.7658

Model 2, validation ROC AUC

0.7168

Model 0, validation ROC AUC

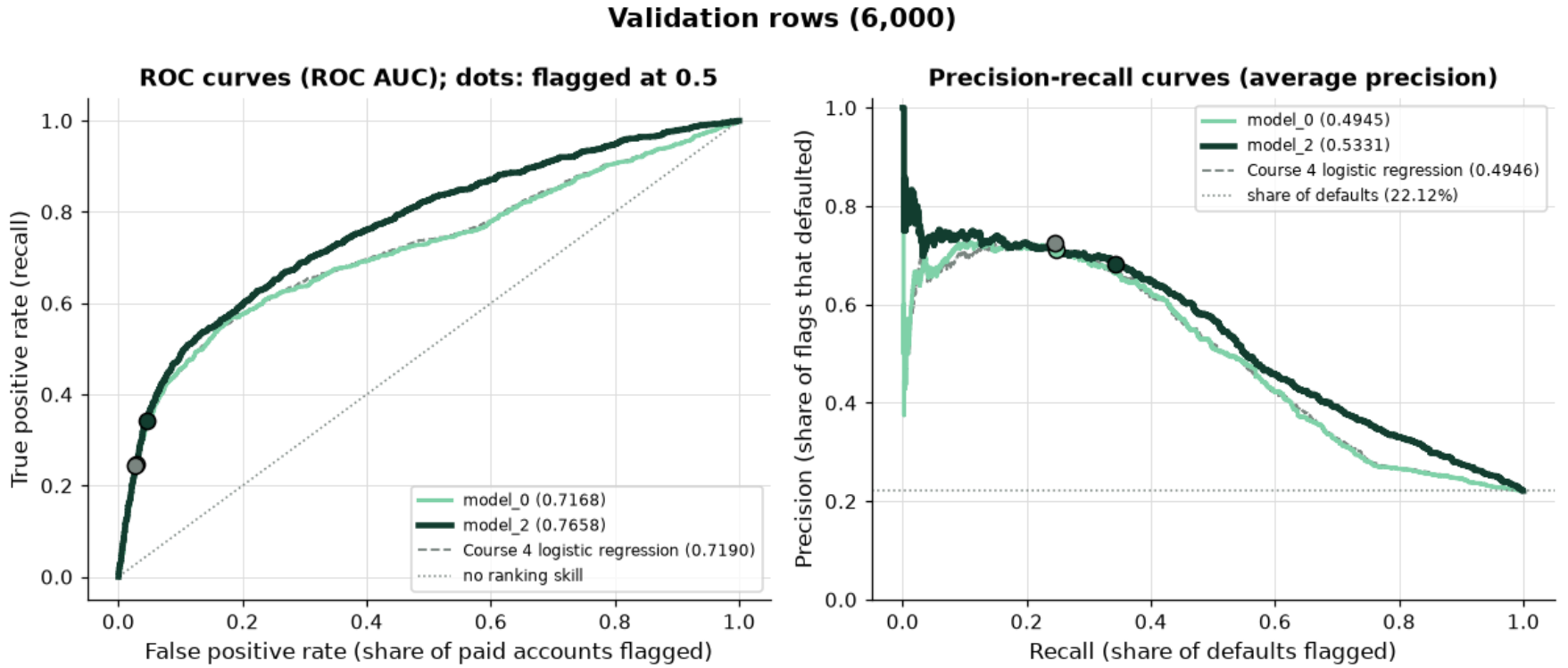
Average precision 0.5331 and 0.4945; Course 4 logistic regression ROC AUC 0.7190.

Day 5

Folds, a threshold, Course 5's LightGBM, and intervals before saying what the numbers support.

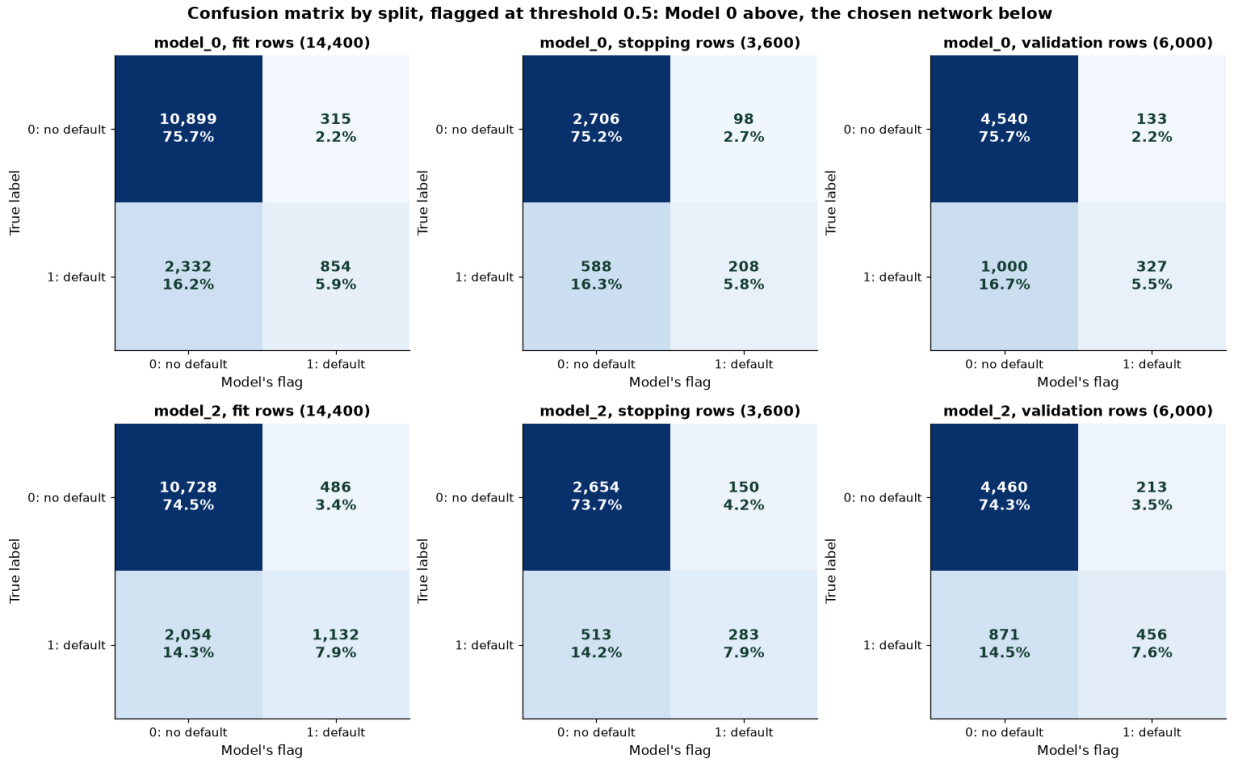
Chosen on the stopping rows; reported once on the validation rows.

Validation, drawn



ROC AUC: Model 2 0.7658, Model 0 0.7168, Course 4 0.7190. Model 0 and Course 4 are nearly one line.

Confusion matrices by split



Recall at 0.5, fit and validation rows: Model 0 26.8% and 24.6%; Model 2 35.5% and 34.4%.

An old name for one unit

F. Rosenblatt, 1958

"The perceptron: A probabilistic model for information storage and organization in the brain."

Psychological Review, volume 65, issue 6, pages 386 to 408

A perceptron is one unit; a multilayer perceptron is units in layers, like Model 5.

Source, read 6 October 2026: the Crossref record of the paper (DOI 10.1037/h0042519). Bibliographic facts only; the paper itself was not read for this course.

Excel side by side

Excel side by side (tested in Excel)

`=MAX(0,A2)  =TANH(A2)  =1/(1+EXP(-A2))`	ReLU, tanh, sigmoid
`=MAX(0,SUMPRODUCT($A2:$B2,$L$2:$M$2)+$N$2)`	ReLU unit 1, in C2, filled down
`=1/(1+EXP(-(SUMPRODUCT(C2:D2,$L$5:$M$5)+$N$5)))`	the output neuron, in E2
`=TANH(SUMPRODUCT($A2:$B2,$L$2:$M$2)+$N$2)`	tanh unit 1, in F2
`=19*32+32+32*1+1`	Model 4: 673 parameters

No Keras or network training in Excel. On 14,400 fit rows: equal to forward_pass within 1e-12, to model.predict within 1e-6.

Save the day: two commits of the log

1. Code

mlkit, ensemblekit, netkit and its tests

2. Model 0

results.csv, first row

3. Model 2

the chosen network added

```
model,rows,roc_auc,average_precision,threshold,accuracy,precision,recall,f1
model_0_one_neuron,validation,0.7168,0.4945,0.5000,0.8112,0.7109,0.2464,0.3660
model_2_16_tanh,validation,0.7658,0.5331,0.5000,0.8193,0.6816,0.3436,0.4569
```

```
git log --oneline -- results.csv      # only the commits that changed the log
```

Threshold 0.5 until day 5. Accuracy beside the all-zero baseline, 0.7788, never to choose.

The AI check

The docstring is the prompt. The code that comes back runs, and its number looks fine.

1

Docstring

Which rows fit, which rows stop, which rows report: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Assert what the docstring promises. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: hidden layers with no activation: 1,185 parameters, stopping loss 0.4702, one neuron's.

Code that runs is not the same as code that is right.

What you can do now

- Compute sigmoid, tanh, and ReLU; read their slopes
- Work a hidden layer by hand, in numpy and Excel
- forward_pass and parameter_count against Keras
- Climb a ladder one setting at a time
- Spot a Dense layer with no activation

Week 1, Day 4 Batch size, epochs, and loss curves